2026 AI 創意設計大賽 決選簡報

AI 於智慧製造領域之應用

拍照即製造：基於 VLM 與 Vibe Coding 之無圖紙化零件重構與自動化生 產系統

打破傳統逆向工程門檻，實現「視覺輸入 → 程式碼生成 → 實體輸出」

參賽組別

智慧製造應用組

作品狀態

佳作推薦 / PoC 實作驗證完成

影片連結

一、問題意識與真實觀察

校園創客空間與中小企業的「無圖紙停機困境」

真實情境觀察

團隊在校園實習工廠與創客空間觀察發現，舊型加工機具常因年代久遠**遺失 3D CAD 圖檔**。當小零件磨損時，必須依賴專業工程師手動量測與繪圖，耗時 2~4 小時，導致設備長期停機。

核心需求：非專業人員能否用「手機拍照」在幾分鐘　內完成備件重構？

2～8 小時前置繪圖

傳統逆向工程門檻高且極度耗時

高額軟硬體成本

昂貴 3D 掃描儀與專用 CAM 軟體授權

依賴老技師經驗

切削與列印參數無法自動傳承與配對

一、創新性與突破

核心創新點: VLM 與 Vibe Coding 單一閉環

跨模態極速製造

零 CAD 繪圖門檻。

單一拍照輸入，AI 自動生成 G-code 實體輸出。

Vibe Coding 參數化 CAD

無須預先建立圖庫。

LLM 直寫 OpenSCAD 腳本，動態生成 3D 結構。

視覺語意材質匹配

自動預測切削參數。

VLM 解析質地，自動 Match 最佳熱床與切削速度。

一、差異化競爭力

傳統逆向工程 vs. 本作品解決方案

比較項目	傳統逆向工程 / 加工流程	本作品解決方案（VLM + Vibe Coding）
前置繪圖門檻	需專業 CAD 工程師，耗時 2～8 小時	**零 CAD 門檻，AI 秒級生成程式碼**
硬體與授權成本	昂貴 3D 掃描儀與高額 CAM 軟體授權	**僅需行動裝置拍照 + LLM API 運作**
材質與切削配置	依賴老技師手動試錯與經驗設定	**AI 資料庫比對，即時輸出最適參數**
極小批量彈性	開模與編程成本過高，難以響應單件需求	**零編程成本，完美實現「即拍即做」備件**

二、作品完成度與實做展示

系統 4 步驟操作流程

STEP 1: 輸入端

實拍待重構零件



STEP 2: AI 解析端

VLM 語意與 JSON 提取

```scad
/*
    Hex Nut - Parametric OpenSCAD Model
    六角螺帽參數化模型

    Reference:
    M10 x 1.5 Hex Nut
    Simplified CAD geometry

    Units: mm
*/

// ============================================
// 1. Parameters
// ============================================

across_flats = 17.0;      // 六角對邊距
nut_height   = 8.0;       // 螺帽總高度

thread_diameter = 10.0;   // 螺紋公稱直徑
thread_pitch    = 1.5;    // 螺距

// Simplified internal hole
hole_diameter = 8.5;

// Chamfer parameters
top_chamfer    = 0.8;
bottom_chamfer = 0.8;

// Through hole
through_hole = true;

// Rendering resolution
$fn = 96;

// ============================================
// 2. Derived Parameters
// ============================================

// Regular hexagon circumradius
corner_radius = across_flats / sqrt(3);

// Radius of the chamfer clipping envelope
envelope_radius = corner_radius + 0.02;

// ============================================
// 3. Hexagonal Prism
// ============================================

module hex_prism() {

    linear_extrude(
        height = nut_height,
        convexity = 10
    )

    polygon(
        points = [
            for (i = [0:5])
                [
                    corner_radius *
                    cos(30 + 60*i),

                    corner_radius *
                    sin(30 + 60*i)
                ]
        ]
    );
}

// ============================================
// 4. Outer Chamfer Envelope
// ============================================

module chamfer_envelope() {

    rotate_extrude(
        angle = 360,
        convexity = 10
    )

    polygon(
        points = [

            // Bottom center
            [0, 0],

            // Bottom chamfer outer radius
            [
                across_flats / 2,
                0
            ],

            // Bottom chamfer transition
            [
                envelope_radius,
                bottom_chamfer
            ],

            // Main body
            [
                envelope_radius,
                nut_height - top_chamfer
            ],

            // Top chamfer transition
            [
                across_flats / 2,
                nut_height
            ],

            // Top center
            [0, nut_height]
        ]
    );
}

// ============================================
// 5. Outer Nut Body
// ============================================

module nut_outer() {

    intersection() {

        hex_prism();

        chamfer_envelope();
    }
}

// ============================================
// 6. Internal Hole
// ============================================

module nut_hole() {

    translate([0, 0, -0.1])

    cylinder(
        h = nut_height + 0.2,
        d = hole_diameter,
        center = false
    );
}

// ============================================
// 7. Complete Nut
// ============================================

module hex_nut() {

    difference() {

        nut_outer();

        if (through_hole) {
            nut_hole();
        }
    }
}

// ============================================
// 8. Render
// ============================================

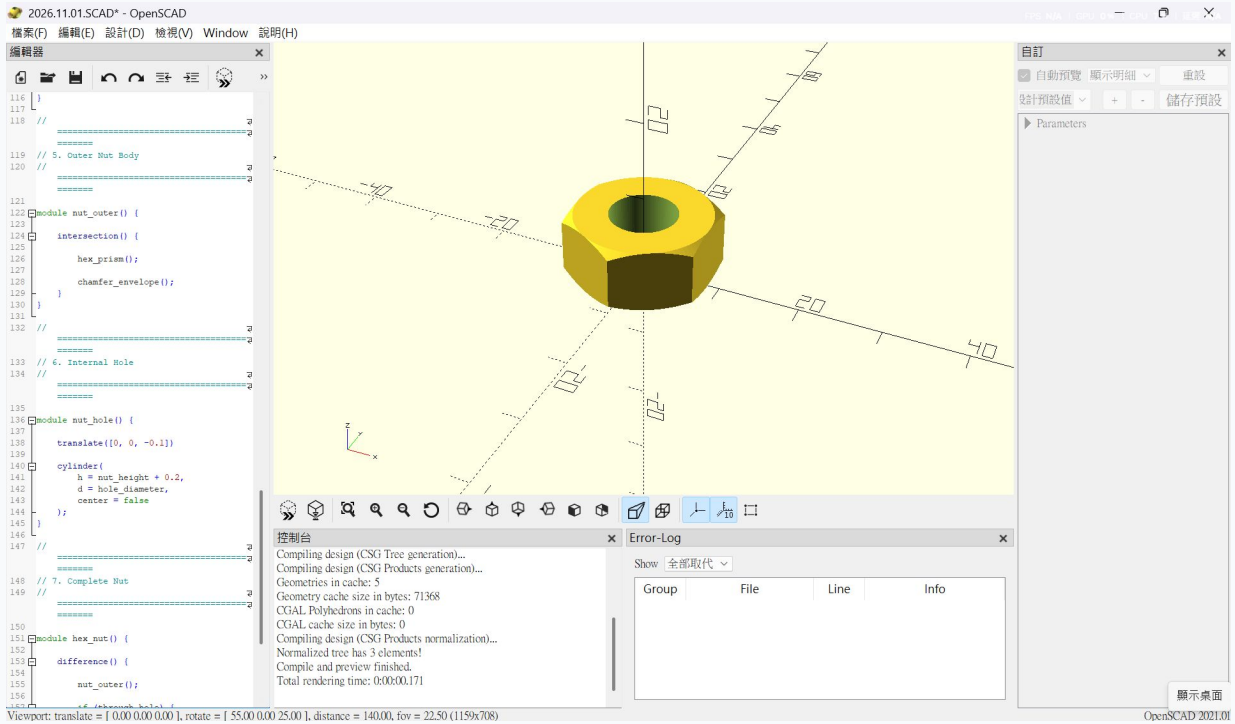
hex_nut();
```

二、作品完成度與實做展示

系統 4 步驟操作流程與畫面截圖

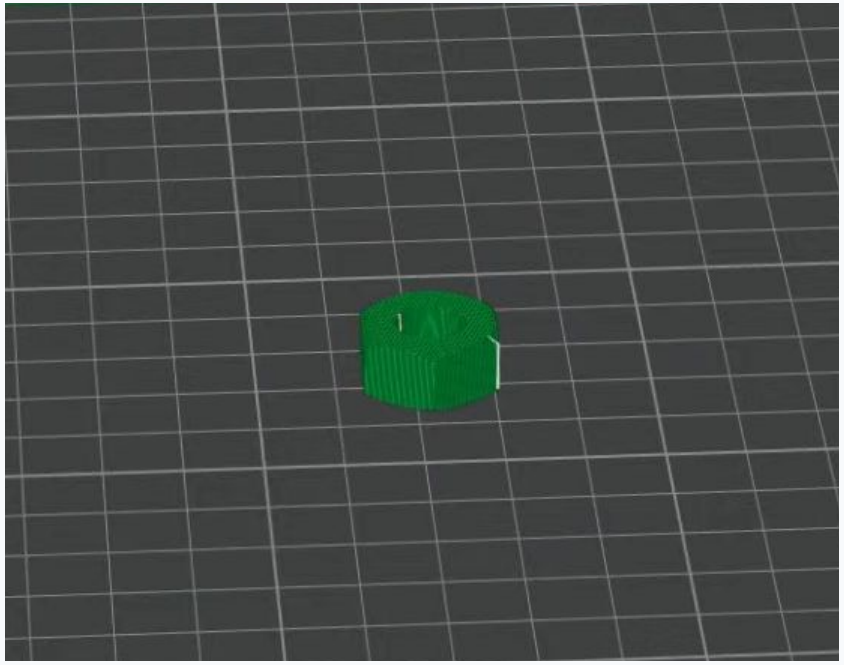
STEP 3: 輸出端

OpenSCAD 3D 渲染



STEP 4: 製造端

Bambu G-code 切片



三、AI 技術應用深度

Prompt 設計策略與實戰迭代對比

初始提示詞（結果不佳）

"請根據這張照片寫出 3D 零件的 OpenSCAD 程式碼。"

遇瓶頸與缺失：

- **幾何尺寸漂移：** AI 無法準確推算孔距與外徑比例。
- **語法錯誤：** 未約束輸出格式，包含大量解釋性文字導致腳本無法編譯。

優化提示詞（CoT + JSON 約束）

"角色：資深 CAM 專家。引導 CoT：【圖像幾何提取】$\rightarrow$【材質估算】$\rightarrow$【JSON 約束】$\rightarrow$【純 OpenSCAD 腳本】"

優化成果：

- **零公差語法崩壞：** 強制輸出 JSON 物件與參數變數，可直刷 3D 列印。
- **自動適應性：** 成功將視覺特徵轉譯為符合物理特性的切削參數。

二、場域驗證與試用紀錄

師生與場域真實試用回饋

「傳統逆向工程需要昂貴的 3D 掃描儀與複雜訓練。這套系統用手機拍照就能產出腳本，大幅降低了實習教學與設備維修的門檻。」

校園實習工廠指導老師
教學與設備維修評價

「我不需要熟練掌握 CAD 繪圖軟體的多重指令，只要下 Prompt 就能修正模型參數，對非機械背景的跨領域新手非常友善！」

創客專題試用同學
跨領域操作體驗

「設備零件損壞時最怕停工等待。現場拍照即印備品，將原本需要外包訂購數天的流程縮短至幾十分鐘。」

設備維修技師
產業落地效益評估

三、AI 技術應用深度

系統技術架構與雙層處理機制

1 Vision Layer (視覺與材質辨識)

- **多模態大模型**: 採用 GPT-4o / Claude 3.5 Sonnet。
- **Zero-shot 材質識別**: 解析孔徑、倒角與幾何特徵。
- **表面質感評估**: 解析金屬/工程塑膠之切削光潔度。

2 Logic Layer (代碼與路徑生成)

- **Vibe Coding 轉譯**: 將非結構化影像轉化為 OpenSCAD 腳本。
- **JSON 結構化約束**: 自動寫入極限公差與密度參數。
- **CAM 切片配對**: 直接轉化為裝備可執行之 G-code 檔。

三、AI 關鍵角色

跨領域轉譯中樞與經驗庫

90%

備件開發與前置作業時間縮短幅度

跨領域語意轉譯者（Bridge）

將非結構化影像精準轉譯為結構化機械控制代碼。

經驗數位化（Knowledge Agent）

取代對老技師切削經驗的依賴，自動匹配最佳列印條件。

二、落地性效益

產業效益與分散式製造展望

時間效益（極速響應）

備件開發前置時間由數天縮短至幾十分鐘，顯著降低工廠停工損失。

成本效益（降低門檻）

降低對高階 CAD/CAM 繪圖員依賴，減輕中小企業人力培訓負擔。

製造彈性（分散供應鏈）

結合 3D 列印與 CNC 邊緣設備，打造雲端「即拍即做」靈活供應鏈。

結語與作品展示

顛覆傳統逆向工程
奠定無圖紙化智慧製造新標竿

透過多模態視覺模型（VLM）與 Vibe Coding 深度整合，成功實現從視覺輸入到實體輸出的端到端單一閉環。

🔓 **已設置權限：「知道連結的任何人皆可檢視」**

1 分鐘操作 Demo 影片

掃描 QR Code 或點擊下方連結觀看
https://youtu.be/c-9FLO4qkz4